

目录

NVDEC SDK 安装指南	1
概述	1
安装状态: ☒ 已完成	1
已安装组件	1
系统环境要求	2
已验证配置	2
GPU 支持	2
安装方法	2
方案一: 从 GitHub 获取 (推荐)	2
方案二: 从 NVIDIA 官网下载	2
方案三: 使用 apt (部分功能)	3
技术方案	3
为什么使用 FFmpeg nv-codec-headers?	3
动态加载机制	3
验证步骤	3
1. 检查头文件	3
2. 编译插件	3
3. 检查编译产物	3
4. 验证符号	3
常见问题	4
Q1: 编译时找不到 nvcuvid.h	4
Q2: 运行时 dlopen 失败	4
Q3: cuGraphicsGLRegisterBuffer 失败	4

NVDEC SDK 安装指南

概述

本文档记录 NVIDIA Video Codec SDK 的安装过程和验证步骤,用于支持 NativeVideoPlugin 的 NVDEC 硬件解码功能。

安装状态: ☒ 已完成

已安装组件

1. FFmpeg nv-codec-headers

- 来源: <https://github.com/FFmpeg/nv-codec-headers>
- 版本: 最新稳定版 (支持 CUDA 11.x - 13.x)
- 位置: Assets/Plugins/NativeVideoPlugin/include/

2. 头文件清单

```
include/
├── dynlink_cuda.h
├── dynlink_cuviddec.h
├── dynlink_loader.h
├── dynlink_nvcuvid.h
├── nvcuvid.h (兼容层)
└── nvEncodeAPI.h
```

3. 动态加载实现

- 使用 dlopen/dlsym 运行时加载 libnvcuvid.so

- 无需编译时链接完整 SDK
- 兼容系统已安装的运行时库

4. 编译状态

- 文件大小: 54KB
 - 无警告和错误
 - 所有 CUID 函数已链接
 - 依赖正确: libcuda, libcudart, libOpenGL
-

系统环境要求

已验证配置

- ✓ CUDA 13.0.88
- ✓ NVIDIA Driver 590.48.01
- ✓ libnvcuvid.so.1 (运行时库)
- ✓ libnvidia-decode-590 (解码库)
- ✓ libnvidia-encode-590 (编码库)
- ✓ OpenGL 4.5+

GPU 支持

- 查看 GPU: nvidia-smi
 - 支持的编解码器: <https://developer.nvidia.com/video-encode-and-decode-gpu-support-matrix-new>
-

安装方法

方案一：从 GitHub 获取（推荐）

```
cd /tmp
git clone --depth 1 https://github.com/FFmpeg/nv-codec-headers.git
cd nv-codec-headers
```

项目本地安装

```
cp -r include/ffnvcodec/* /home/zby/RoboMasterClientWMJ/Assets/Plugins/NativeVideoPlugin/include/
```

方案二：从 NVIDIA 官网下载

1. 访问 NVIDIA 开发者网站

<https://developer.nvidia.com/nvidia-video-codec-sdk>

2. 登录并下载

推荐版本: **Video Codec SDK 12.2**

- 支持 CUDA 11.x - 13.x
- 兼容 Ubuntu 20.04+

3. 安装头文件

```
cd ~/Downloads
unzip Video_Codec_SDK_12.2.72.zip
cd Video_Codec_SDK_12.2.72
```

```
# 安装到项目
cp -r Interface/* /home/zby/RoboMasterClientWMJ/Assets/Plugins/NativeVideoPlugin/include/
```

方案三：使用 apt（部分功能）

```
sudo apt install nvidia-cuda-dev
find /usr -name "nvcuvid.h" 2>/dev/null
```

技术方案

为什么使用 FFmpeg nv-codec-headers?

特性	FFmpeg 方案	NVIDIA 官方 SDK
获取方式	Git 直接获取	需注册账号下载
许可	开源免费	需同意许可协议
链接方式	动态加载	静态链接
兼容性	支持多版本运行时	版本绑定

动态加载机制

```
// 1. 加载库
void* handle = dlopen("libnvcuvid.so.1", RTLD_LAZY);

// 2. 获取函数指针
cuvidCreateVideoParser = (tcuvidCreateVideoParser*)dlsym(handle, "cuvidCreateVideoParser");

// 3. 直接调用
cuvidCreateVideoParser(&parser, &params);
```

优势： - 运行时库可以随驱动升级 - 编译时不依赖完整 SDK - 支持不同 GPU 使用不同版本

验证步骤

1. 检查头文件

```
ls -la /home/zby/RoboMasterClientWMJ/Assets/Plugins/NativeVideoPlugin/include/
```

2. 编译插件

```
cd Assets/Plugins/NativeVideoPlugin
./build.sh
```

3. 检查编译产物

```
ls -lh /home/zby/RoboMasterClientWMJ/Assets/Plugins/x86_64/libNativeVideoPlugin.so
```

4. 验证符号

```
nm -D Assets/Plugins/x86_64/libNativeVideoPlugin.so | grep -E "cuvidCreate|nvp_"
```

预期输出：

```
U cuvidCreateVideoParser
0000000000003abc T nvp_init
0000000000003def T nvp_push_udp
```

常见问题

Q1: 编译时找不到 nvcuvid.h

解决: 确保头文件已复制到 include/ 目录, 并在 CMakeLists.txt 中添加:

```
target_include_directories(NativeVideoPlugin PRIVATE ${CMAKE_CURRENT_SOURCE_DIR}/include)
```

Q2: 运行时 dlopen 失败

解决:

检查运行时库

```
ldconfig -p | grep nvcuvid
```

如果没有, 安装:

```
sudo apt install libnvidia-decode-XXX # XXX 为驱动版本
```

Q3: cuGraphicsGLRegisterBuffer 失败

原因: 在非渲染线程调用 OpenGL 互操作

解决: 延迟 GL 资源初始化到渲染线程 (参见原生视频插件开发计划)

文档更新日期: 2026年2月9日